Student Performance Analysis

A PROJECT REPORT

SUBMITTED BY

Roll Number	Registration Number	Student Code	Student Name
24103460090	24103002871	BWU/BCA/24/094	Sayantan Jana

Of in partial fulfilment for the award of the degree

BACHELOR OF COMPUTER APPLICATION

IN

DEPARTMENT OF COMPUTATIONAL SCIENCE



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

INDEX

❖ ABSTRACT

Sl	Content	Page No.
1.	INTRODUCTION	1
2.	LITERATURE SURVEY	2
3.	SYSTEM ANALYSIS AND DESIGN	3
4.	MEDHODOLOGY	4
5.	IMPLEMENTATION	7
6.	TESTING AND DEBUGGING	14
7.	CONCLUSION	21
8.	FUTURE ENHANCEMENT	22
9.	REFERENCES	23

ABSTRACT

This project presents a comprehensive machine learning pipeline for analysing and predicting student academic performance using the Student Performance dataset from the UCI repository (Portuguese secondary school students). The primary objective is to understand the factors influencing final grades and to predict students' future performance, specifically estimating a fourth-grade indicator (G4_Predicted) based on historical academic and demographic attributes.

The methodology follows a structured workflow: data acquisition via Kaggle, preprocessing (handling missing values, duplicates, categorical encoding), exploratory data analysis to identify correlations, feature selection based on correlation thresholds, model training and evaluation, visualization, and final prediction. Four regression models were implemented and compared - Linear Regression, Logistic Regression, Decision Tree Regressor, and Random Forest Regressor - using metrics including Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 score to determine optimal performance.

Exploratory analysis reveals strong correlations between period grades (G1, G2) and final grade (G3), confirming prior academic performance as the dominant predictor. Feature selection identifies key influential variables while excluding weaker correlates. Among the evaluated models, ensemble methods demonstrate superior predictive capability. The pipeline culminates in generating predictions for all students, producing a G4_Predicted score using a Random Forest model trained on non-final-grade features.

The results provide insights into academic performance drivers and offer a scalable framework for early identification of students who may benefit from academic support. By predicting future performance indicators, this system can assist educators in implementing targeted interventions. The modular design allows for extension to different datasets and prediction tasks, making it a practical tool for educational data mining and academic performance forecasting

1. INTRODUCTION

Education plays a vital role in the overall development of individuals and society. A student's academic performance is often considered a key indicator of their knowledge, skills, and future opportunities. With the rapid growth of technology in the education sector, a large amount of student-related data is being generated daily. This has led to the rise of Educational Data Mining (EDM), a field that uses data mining and machine learning techniques to extract meaningful insights from educational data and improve learning outcomes.

Traditional methods of evaluating student performance rely heavily on final examinations, which do not provide early warnings about a student's academic progress. Numerous factors, such as family background, study habits, social life, past grades, and school-related attributes, influence a student's performance. Without analyzing these factors together, it becomes difficult to identify at-risk students and provide timely academic support. Therefore, there is a need for a data-driven approach to analyze these factors and predict a student's future academic performance.
The dataset used for this project is the "Student Performance" dataset obtained from the UCI Machine Learning Repository via Kaggle. It contains information about secondary school students from two Portuguese schools. The dataset consists of various attributes related to the student's demographics, family background, social activities, school support, and academic performance. The academic data is represented by three grade periods, namely G1 (First Period Grade), G2 (Second Period Grade), and G3 (Final Grade). For the purpose of this project, G3 is used as the target variable for model training and evaluation, while G4_Predicted is generated as an estimated future grade.

This project holds great significance in the field of education. By predicting student performance in advance, educational institutions can identify students who are at risk of underperforming and provide them with necessary guidance, counselling, or remedial classes at an early stage.
Furthermore, it helps educators understand the key factors affecting academic success and make data-driven decisions to improve teaching strategies. The modular pipeline developed in this project is also reusable, making it adaptable for similar datasets in the future.

This report is organized into five chapters. Chapter 1 gives an overview of the study, including its background, objectives, scope, and significance. Chapter 2 reviews existing literature related to student performance prediction. Chapter 3 explains the methodology adopted, covering data collection, preprocessing, feature selection, model implementation, and evaluation. Chapter 4 presents the results obtained along with their discussion. Finally, Chapter 5 concludes the report by summarizing the key findings and suggesting possible improvements for future work.

2. LITERATURE SURVEY

Many researchers have explored the use of data mining and machine learning techniques to predict student academic performance under the domain of Educational Data Mining (EDM). One of the most influential studies was conducted by Cortez and Silva (2008), who used the same UCI Student Performance dataset to predict final grades using classification algorithms such as Decision Trees, Naïve Bayes, and Support Vector Machines. Their findings revealed that prior academic performance, particularly G1 (first period grade) and G2 (second period grade), were the strongest predictors of the final grade (G3), while social and demographic factors had comparatively less impact. This study not only validated the usability of this dataset, but also established it as a benchmark for future research in this field.

A broader perspective on EDM was provided by Romero and Ventura (2010) through their comprehensive survey of existing literature. They categorized EDM applications into prediction, clustering, relationship mining, and visualization, highlighting that predictive modeling is one of the most effective approaches for identifying at-risk students and enabling timely academic interventions. Similarly, Kotsiantis et al. (2004) applied machine learning techniques to predict student performance in distance learning environments and concluded that the use of multiple attributes, along with appropriate algorithms, significantly improved prediction accuracy. These early studies laid the foundation for applying intelligent systems to enhance educational outcomes.

With the advancement of machine learning, more recent studies have shifted their focus towards ensemble learning methods for better accuracy and robustness. Al-Barrak and Al-Razgan (2016) utilized the J48 Decision Tree algorithm to predict students' final GPA and successfully extracted interpretable rules to identify academically weak students. Likewise, Hussain et al. (2018) compared several classification and regression models and found that the Random Forest algorithm consistently outperformed others due to its ability to handle non-linear relationships and reduce overfitting. Furthermore, Yassein et al. (2017) emphasized that regression-based approaches are far more practical than multi-class classification when predicting exact grade values, as they provide more precise and actionable results for educational planning.

Despite the vast amount of research conducted in this area, most existing studies primarily focus on predicting the final grade (G3) or simply classifying students as pass or fail. Very few have attempted to generate a projected future grade by excluding the final grade from the training features to simulate real-world prediction scenarios. Taking inspiration from these existing works, the present study builds upon the findings of Cortez and Silva (2008) by implementing a regression-based pipeline that compares multiple models, including Linear Regression, Logistic Regression, Decision Tree Regressor, and Random Forest Regressor. Instead of solely predicting G3, this project aims to generate a continuous estimated grade, referred to as G4_Predicted, using only prior academic and non-final attributes. This proactive approach aligns more closely with the core goal of Educational Data Mining, which is to detect and support struggling students before their final academic evaluation.

3. SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

3.1.1 Existing System

In most educational institutions, student performance is evaluated through traditional methods such as class tests, assignments, attendance and final examinations. This evaluation process is mostly reactive, as it only identifies a student's performance after completion of exams. Moreover, it fails to consider various influencing factors like family background, study time, social activities, past academic records and other personal attributes. As a result, timely identification and support for academically weak students become difficult.

3.1.2 Proposed System

The proposed system is a machine learning-based "Student Performance Prediction System". It automates the entire process of analyzing student data, identifying the factors affecting academic performance and predicting a student's future grade. Instead of merely classifying students as pass or fail, this system uses regression techniques to generate a continuous predicted grade, referred to as "G4_Predicted". By excluding the final grade (G3) while training the model, the system mimics a real-time scenario where future performance is predicted using only prior academic records andnon-academic factors. This helps in the early detection of at-risk students and enables the institution to take necessary remedial measures in time.

3.1.3 Feasibility Study

A feasibility study is carried out to determine whether the proposed system is practical and worth implementing.

3.1.4 Technical Feasibility

The system is developed using Python, which is an open-source, platform-independent programming language. The required libraries such as Pandas, NumPy, Matplotlib, Seaborn and Scikit-learn are freely available and well-suited for data analysis and machine learning tasks. Hence, the project is technically feasible.

3.1.3.5 Operational Feasibility

The proposed system is simple, user-friendly and can be executed with minimal technical knowledge. By just running the main.py file, the entire pipeline executes automatically from data preprocessing to prediction. The output is generated in a readable format along with visual representations, making it easy for educators to understand and use. Thus, the system is operationally feasible.

3.2 Requirement Analysis

3.2.1 Functional Requirements

The functional requirements define the operations that the system must perform. They are as follows:

- Data Acquisition: The system must load the Student Performance dataset in CSV format.
- Data Preprocessing: The system must handle missing values, duplicate records and convert categorical data into numerical form using Label Encoding.

- Exploratory Data Analysis (EDA): The system must analyze and visualize the data to find patterns and relationships between different attributes and student grades.
- Feature Selection: The system must identify and select the most significant features using Pearson's correlation technique.
- Model Building: The system must implement and train multiple regression models such as Linear Regression, Logistic Regression, Decision Tree Regressor and Random Forest Regressor.
- Model Evaluation: The system must evaluate and compare each model using MAE, MSE, RMSE and R^2 Score.
- Prediction: The system must generate G4_Predicted for all students using the best-performing model.
- Visualization: The system must display graphs like correlation heatmaps, actual vs predicted plots, residual plots and feature importance graphs.
- Result Export: The system must save the final predicted results in a CSV file named predicted_G4_results.csv.

3.3 System Design

3.3.1 System Architecture

The system follows a modular pipeline architecture. The entire process is controlled by main.py, which sequentially calls each module in a predefined order. The architecture begins with the dataset as input, passes it through various stages of preprocessing, analysis, modelling and finally produces predicted results along with visual outputs. This design ensures a smooth flow of data from one stage to another with minimal dependency issues.

3.3.2 System Flowchart

The step-by-step flow of the proposed system is as follows:

1. Start – Initialize the pipeline by running main.py.
2. Dataset Loading – Locate and load the student performance CSV file.

3. Data Preprocessing – Clean the data by removing missing values, duplicates and encoding categorical attributes.

4. Exploratory Data Analysis (EDA) – Generate statistical summaries and visual plots to understand the dataset.

5. Feature Selection – Select relevant features based on their correlation with the target variable (G3).

6. Model Training and Evaluation – Split the data into training and testing sets, train all four regression models and compare their performance.

7. Best Model Selection – Identify the model with the highest R^2 score and lowest error values.

8. Prediction – Train the Random Forest Regressor on selected features (excluding G3) and predict G4_Predicted for all students.

9. Visualization – Display model comparisons, residual plots, feature importance and correlation heatmaps.

10. Export Results – Save the final output as predicted_G4_results.csv.

11. End – Pipeline execution completed successfully.

3.3.4 Input and Output Design

Input Design: The only input required by the system is the Student Performance dataset in CSV format. The system automatically detects the dataset from the local directory or from the KaggleHub cache.

Output Design: The system produces both on-screen and file-based outputs. On-screen outputs include preprocessing summaries, EDA statistics, model evaluation metrics, feature selection results and prediction summaries. File-based outputs include predicted_G4_results.csv containing Student_ID, school, sex, age, G1, G2, G3 and G4_Predicted, along
with correlation_heatmap.png saved during visualization.

3.4 Tools and Technologies Used

- Programming Language: Python 3.x
- Data Manipulation: Pandas and NumPy
- Data Visualization: Matplotlib and Seaborn
- Machine Learning: Scikit-learn
- Dataset Source: KaggleHub (UCI Student Performance Dataset)
- Development Environment: Visual Studio Code
- Operating System: Windows

3.5 Summary

This chapter dealt with the system analysis and design of the Student Performance Prediction System. The existing system was analyzed and the proposed system was designed to overcome its limitations. Functional and non-functional requirements were identified, followed by the system architecture, flowchart, module design and input-output design. The use of a modular, regression-based approach ensures accurate prediction of G4_Predicted, early identification of weak students and better decision-making for academic improvement.

4. METHODOLOGY

4.1 Overview

This chapter describes the systematic approach adopted to develop the Student Performance Prediction System. The methodology involves a sequence of steps including data collection, data preprocessing, exploratory data analysis, feature selection, model building, evaluation, prediction and visualization. The entire process is implemented in a modular form using Python and the Scikit-learn library, and is automated through the main.py file.

4.2 Data Collection

The dataset used for this project is the "Student Performance" dataset obtained from the UCI Machine Learning Repository via Kaggle. It was downloaded using the kagglehub package in the Student.py module. The dataset consists of student information from two Portuguese secondary schools, namely Gabriel Pereira (GP) and Mousinho da Silveira (MS). It contains 649 instances and 33 attributes that cover demographic, social, family-related and academic aspects of students. The dataset includes three grade variables: G1 (first period grade), G2 (second period grade and G3 (final grade), all graded on a scale from 0 to 20.

4.3 Data Preprocessing

Data preprocessing is one of the most crucial steps in any machine learning pipeline, as raw data often contains inconsistencies that can affect model performance. The following steps are carried out in the preprocessing.py module:

4.3.1 Data Loading

The load_data() function reads the CSV file using the Pandas read_csv() method and converts it into a DataFrame for further processing.

4.3.2 Handling Missing Values

The remove_missing_values() function checks for null values using isnull().sum() and drops rows containing missing values using dropna(). This ensures the dataset is complete and free from inconsistencies.

4.3.3 Removal of Duplicates

Duplicate records can lead to biased results during model training. The remove_duplicates() function removes identical rows using drop_duplicates() and displays the number of duplicate rows removed.

4.3.4 Categorical Encoding

The dataset contains several categorical attributes such as school, sex, address, famsize, Mjob, Fjob, reason, guardian and others. Since machine learning algorithms require numerical input, these attributes are converted to numeric form using LabelEncoder() from Scikit-learn.
The encode_categorical() function performs this encoding for all columns with object data type.

4.3.5 Data Preparation

After cleaning and encoding, the data is ready for analysis and modelling. The preprocess() function integrates all the above steps and returns the cleaned DataFrame.

4.4 Exploratory Data Analysis (EDA)

Exploratory Data Analysis is performed using the eda.py module to understand the underlying structure of the dataset, detect patterns, identify outliers and examine relationships between variables. The following visualizations and analyses are carried out:

4.4.1 Statistical Summary

Basic statistics such as count, mean, standard deviation, minimum, maximum and quartiles are generated using describe() to get an overview of numerical attributes.

4.4.2 Correlation Heatmap

A correlation matrix is generated for all numerical features using Pearson's correlation coefficient. The plot_correlation_heatmap() function visualizes this using Seaborn's heatmap() to identify highly correlated variables, especially with G1, G2 and G3.

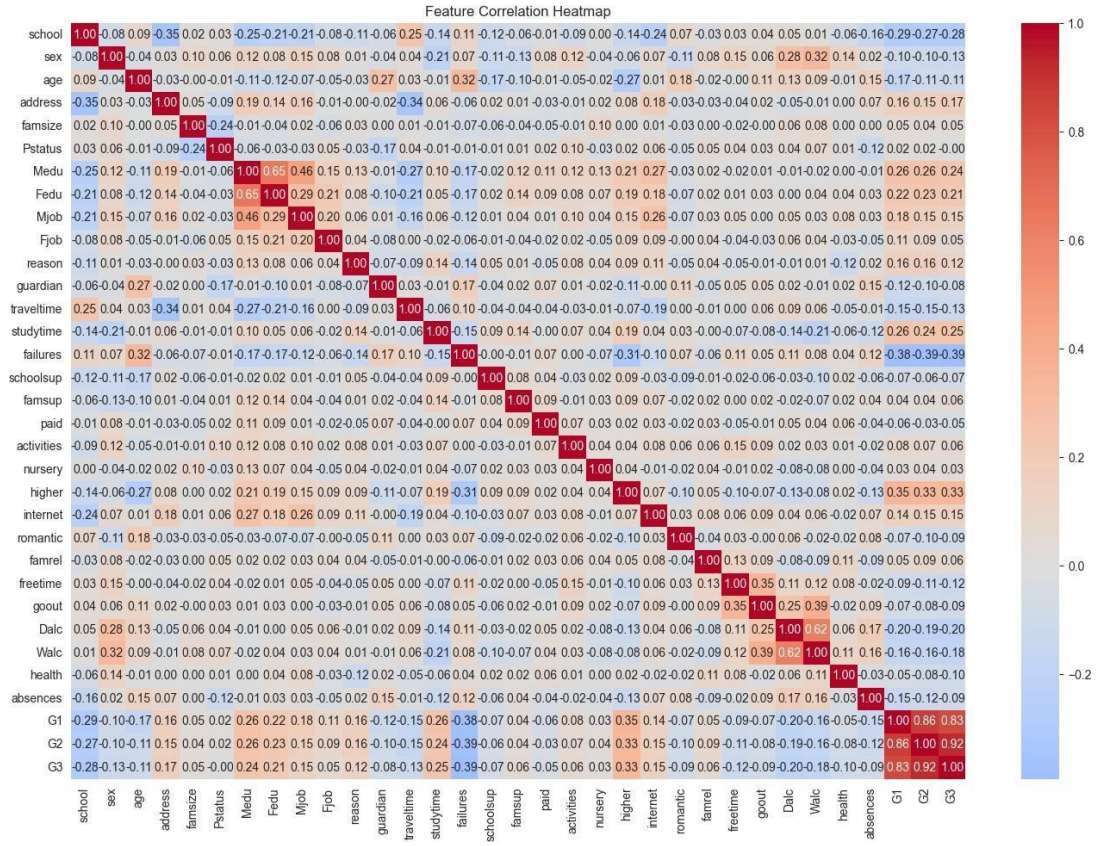


Fig. 4.1

4.4.3 Target Distribution

The distribution of the target variable (G3) is plotted using a histogram with a Kernel Density Estimate (KDE) through the plot_target_distribution() function to check its spread and skewness.

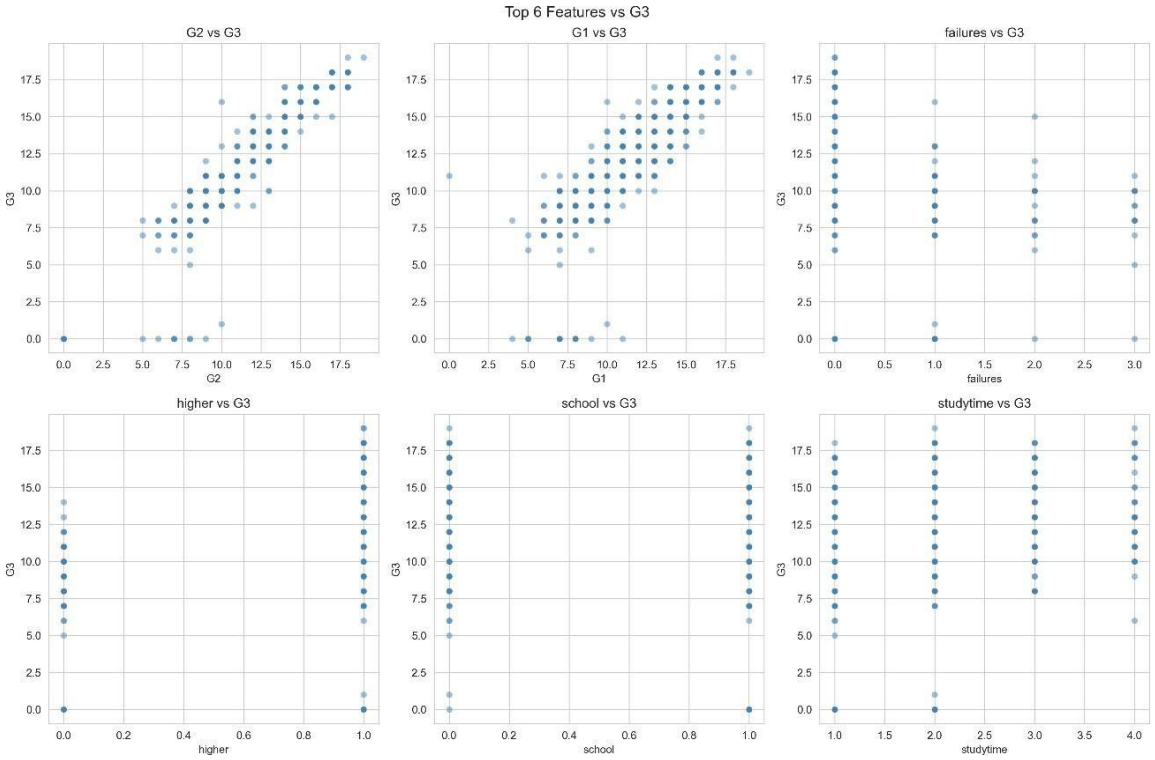


Fig. 4.2

4.4.4 Feature Distribution

Histograms and KDE plots are generated for all numerical features using plot_feature_distributions() to understand their individual distributions.

4.4.5 Categorical Analysis

Categorical features are analyzed using boxplots and bar plots with the help of plot_bar_categorical() to examine their influence on student performance.

4.4.6 Pairplot Analysis

A pairplot is created using plot_pairplot() to visualize pairwise relationships between key numerical attributes, providing a better understanding of interdependencies.

4.5 Feature Selection

Feature selection is performed in the feature_selection.py module to identify the most relevant attributes that have a significant impact on the target variable. Since using all features can increase complexity and may lead to overfitting, only the most influential ones are selected.

4.5.1 Correlation with Target

The correlation_with_target() function computes Pearson's correlation coefficient between each numerical feature and the target variable (G3). Features are then sorted based on the absolute value of their correlation to determine their strength of influence.

4.5.2 Feature Selection Criteria

The select_features() function selects features with an absolute correlation value of 0.1 or greater with the target variable. Additionally, a predefined exclude list (Medu, Fedu, Dalc, Walc) is applied to remove certain features, while an include list (health) is used to retain features that may be relevant despite slightly lower correlation. The final selected features are used for training the regression models.

4.6 Model Building and Evaluation

Model building is carried out in the model.py module. The goal is to implement, train and compare multiple regression algorithms to identify the best-performing model for predicting student performance.

4.6.1 Data Splitting

The dataset is divided into training and testing sets using the train_test_split() function from Scikit-learn. An 80:20 ratio is used, where 80% of the data is used for training and 20% for testing. A fixed random state (random_state=42) is set to ensure reproducibility.

4.6.2 Models Used

The following regression models are implemented and compared in this project:

- Linear Regression: A fundamental regression algorithm that establishes a linear relationship between independent variables and the target variable.
- Logistic Regression: Although traditionally used for classification, it is implemented as a regression-like approach in this pipeline for comparison purposes.
- Decision Tree Reposer: A non-linear model that splits the data based on feature values to make predictions. It captures complex relationships but can be prone to overfitting.
- Random Forest Regressor: An ensemble learning method that builds multiple decision trees and combines their outputs to improve accuracy and reduce overfitting. It is set with n_estimators=100 and random_state=42.

4.6.3 Model Evaluation Metrics

To assess and compare the performance of each model, the following metrics are used:

- Mean Absolute Error (MAE): Measures the average absolute difference between actual and predicted values. It gives a clear idea of the average error magnitude.
- Mean Squared Error (MSE): Calculates the average of squared differences between actual and predicted values. It penalizes larger errors more heavily.
- Root Mean Squossed Error (RMSE): The square root of MSE, which is expressed in the same units as the target variable (grades), making it easier to interpret.
- R^2 Score (Coefficient of Determination):&](...) Represents the proportion of variance in the target variable that is explained by the independent variables. A higher R^2 score indicates better predictive accuracy.

4.6.4 Model Comparison

The train_and_evaluate() function trains all four models, generates predictions on the test set and computes all the evaluation metrics. The results are then displayed in a comparison table, and the best-performing model is identified based on the highest R^2 score and lowest error values.

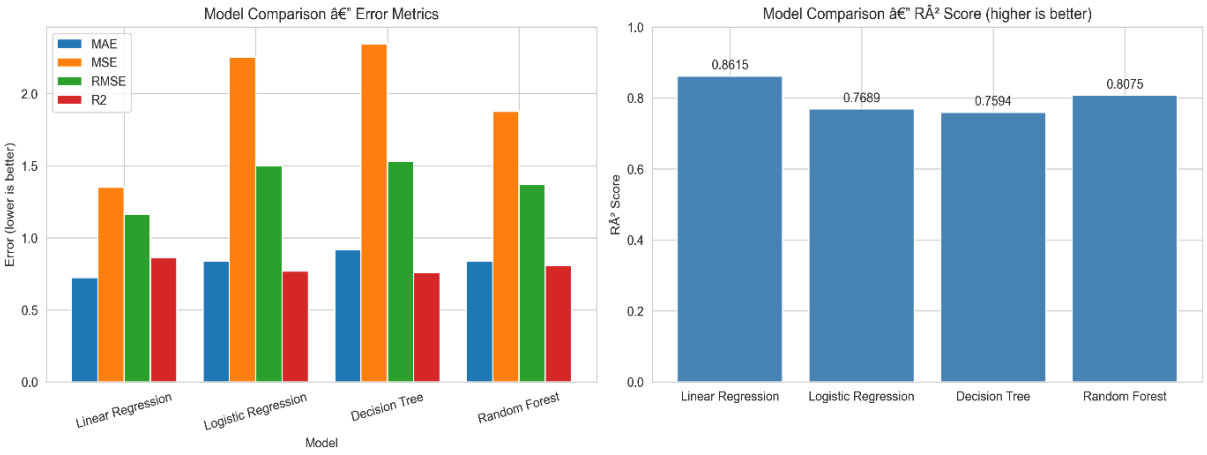


Fig. 4.3

4.7 Prediction and Visualization

4.7.1 Prediction of G4_Predicted

After identifying the best model, prediction is performed using the prediction.py module. Unlike model training where G3 was used as the target, this step simulates a real-world scenario where future performance is predicted without knowing the final grade. For this purpose, a Random Forest Regressor is trained using features that exclude G3 and any existing G4. The model is then applied to all students to generate a new column, G4_Predicted, which represents the estimated grade. The predicted results are printed on screen and saved as predicted_G4_results.csv` in the working directory.

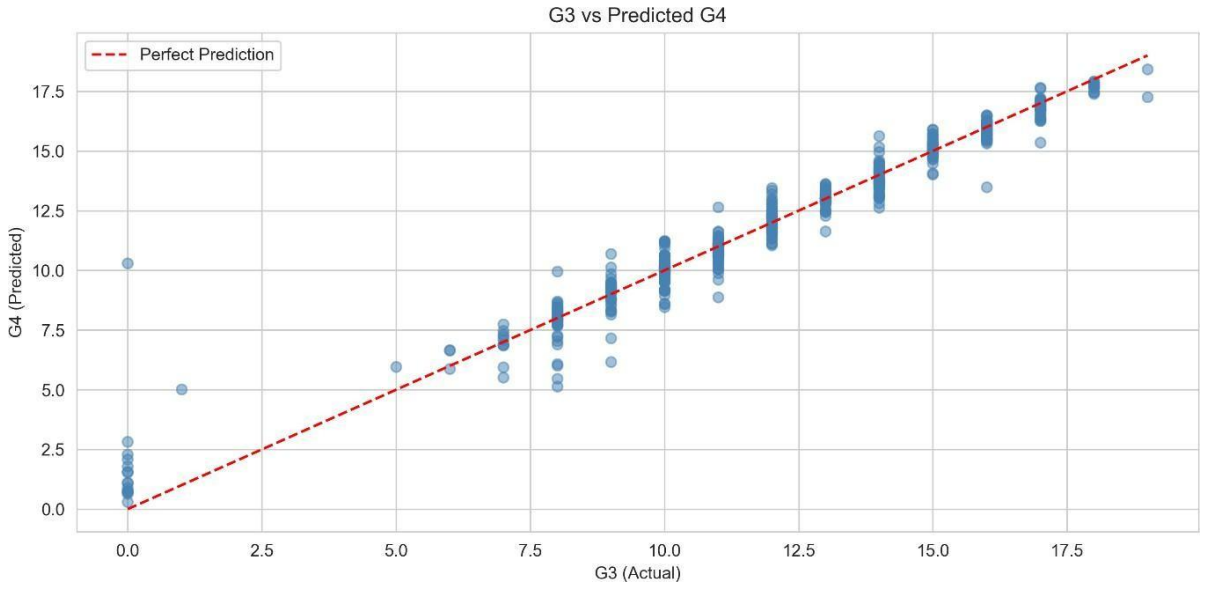


Fig. 4.4

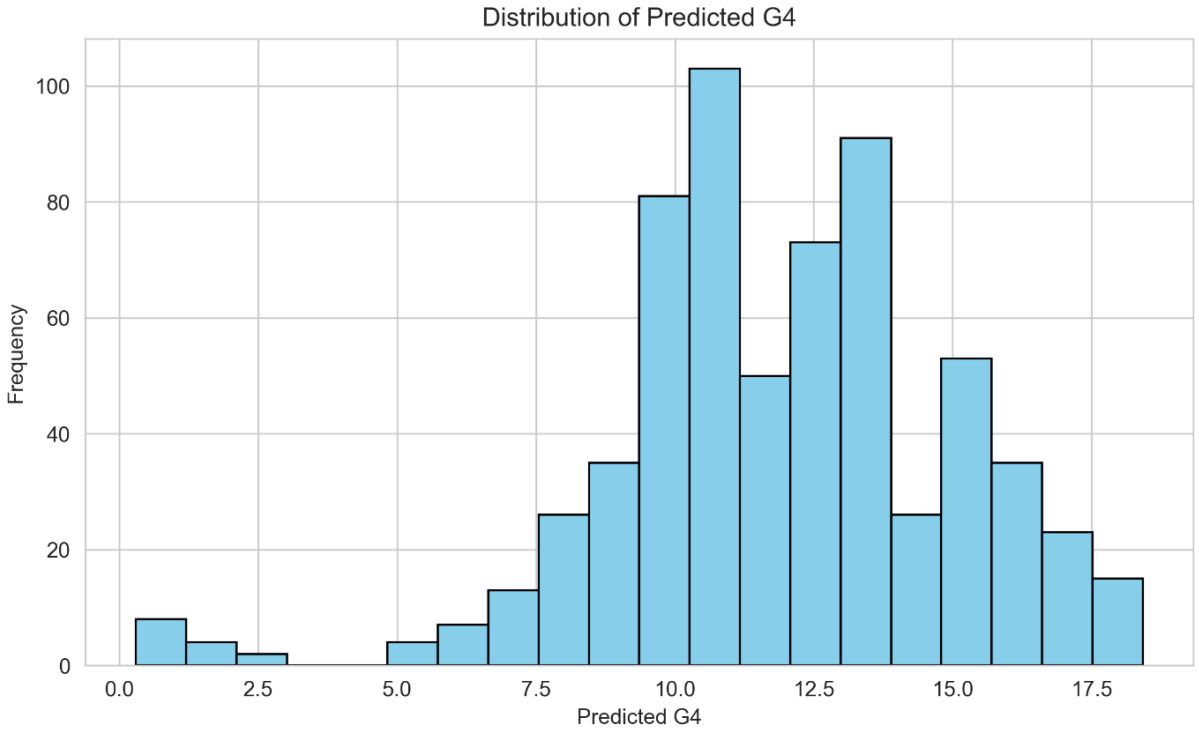


Fig 4.5

4.7.2 Visualization of Results

Visualization is carried to make the findings more interpretable. The visualization.py module creates the following plots:

- Model Comparison Plots: Bar charts displaying MAE, MSE, RMSE and R^2 score for all models to visually compare their performance.
- Actual vs Predicted Plots: Scatter plots with a perfect prediction line to examine how closely the predicted values align with actual values.
- Residual Plots: Scatter plots of residuals (difference between actual and predicted values) versus predicted values to assess model suitability and detect patterns.
- Feature Importance Plot: For the Random Forest Regressor, a bar chart is generated showing the relative importance of each selected feature in making predictions.
- Target vs Features Plots: Scatter plots of the top correlated features with G3 to visualize relationships.
- Correlation Heatmap: A comprehensive heatmap is saved as correlation_heatmap.png to display correlations between all numerical features.

4.8 Pipeline Automation using main.py

The entire methodology is integrated and automated through the main.py file. It acts as the main controller and performs the following sequence:

1. Sequential Execution: Runs preprocessing, EDA, feature selection, model training and evaluation, visualization and prediction in order. 3 Data Handling for Prediction: Re-loads and preprocesses the data fresh before the prediction step, ensuring consistency.

2. Plot Management: Close plots` between steps to prevent them from being stacked, giving a clear visual output.

3. Summary Display: Prints a completion message indicating that the pipeline has been successfully executed and the results have been saved.

4.9 Summary

This chapter has explained the methodology adopted for the Student Performance Prediction System in detail. The steps involved include data collection, preprocessing, exploratory data analysis, feature selection, model implementation, evaluation, prediction and visualization. By using a regression-based approach and excluding the final grade (G3) from the prediction features, the system realistically predicts the future grade (G4_Predicted) for each student. The use of multiple models, proper evaluation metrics and extensive visualization ensures that the system is accurate, reliable and useful for early identification of at-risk students.

5. IMPLEMENTATION

This chapter covers the implementation of the Student Performance Prediction System. It outlines the hardware and software requirements, module-wise implementation, project execution, and the outputs generated.

5.1 Requirements

5.1.1 Hardware Requirements

- Processor: Intel Core i3 or higher
- RAM: 4 GB minimum (8 GB recommended)
- Storage: At least 10 GB of free space
- Input/Output: Keyboard, Mouse and Monitor

5.1.2 Sofware Requirements

- Operating System: Windows 10/11 or Linux
- –Programming Language: Python 3.x
- IDE: Visual Studio Code
- Libraries Used: Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn and KaggleHub
- Dataset: UCI Student Performance Dataset (Downloaded via KaggleHub)

5.2 Module Implementation

The system is divided into the following modules, all integrated and automated by main.py:

- Student.py - Downloads the dataset from Kaggle using kagglehub and displays a basic overview of its structure.
- preprocessing.py - Loads, cleans (removes missing values and duplicates) and encodes categorical features using LabelEncoder().
- eda.py - Performs exploratory analysis through correlation heatmaps, histograms, boxplots, pairplots and target distribution plots.
- feature_selection.py - Selects features with |correlation| >= 0.1 with G3, applying predefined exclude and include lists.
- model.py - Implements and evaluates Linear Regression, Logistic Regression, Decision Tree Regressor and Random Forest Regressor using MAE, MSE, RMSE and R^2 Score with an 80:20 train-test split.
- visualization.py - Generates model comparison charts, residual plots, feature importance

plots, target vs feature plots and saves a correlation heatmap.

- prediction.py - Trains a Random Forest Regressor excluding G3 to predict G4_Predicted for all students and exports the results to predicted_G4_results.csv.

5.3 Execution of the Project

1. Install Dependencies: Install the required libraries using the following command:

   pip install pandas numpy matplotlib seaborn scikit-learn kagglehub

2. Run the Project: Open the folder in VS Code and run main.py

3. View Results: All metrics, plots and summaries will be displayed,
   with predicted_G4_results.csv and correlation_heatmap.png saved in the project directory.

5.4 Output

```
============================================================

STEP 1: PREPROCESSING

============================================================

Data loaded successfully.

Shape before preprocessing: (649, 33)

Missing values removed: 0

Duplicate rows removed: 0

Shape after preprocessing: (649, 33)

Columns: ['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu', 'Fedu', 'Mjob', 'Fjob', 'reason',
'guardian', 'traveltime', 'studytime', 'failures', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery',
'higher', 'internet', 'romantic', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1',
'G2', 'G3']

============================================================

STEP 3: FEATURE SELECTION

============================================================

========== FEATURE SELECTION ==========

Shape: (649, 33)

========== Correlation with G3 ==========

G2          0.918548
```

G1 0.826387

failures -0.393316

higher 0.332172

school -0.284294

studytime 0.249789

Medu 0.240151

Fedu 0.211800

Dalc -0.204719

Walc -0.176619

Name: G3, dtype: float64

Selected features: ['G2', 'G1', 'failures', 'higher', 'school', 'studytime', 'address', 'internet', 'Mjob', 'sex', 'traveltime', 'reason', 'freetime', 'age', 'health']

==========

STEP 4: MODEL TRAINING & EVALUATION

==========

========== MODEL TRAINING & EVALUATION ==========

Features used: ['G2', 'G1', 'failures', 'higher', 'school', 'studytime', 'address', 'internet', 'Mjob', 'sex', 'traveltime', 'reason', 'freetime', 'age', 'health']

Target: G3

Training set: 519 samples

Testing set: 130 samples

========== MODEL COMPARISON ==========

Model	MAE	MSE	RMSE	R2
Linear Regression	0.725031	1.350402	1.162068	0.861522
Logistic Regression	0.838462	2.253846	1.501282	0.768877
Decision Tree	0.915385	2.346154	1.531716	0.759411
Random Forest	0.835846	1.877088	1.370068	0.807512

Best Model: Linear Regression (R^2 = 0.8615)

==========

STEP 6: PREDICTION

==========

========== PREDICT G4 FOR ALL STUDENTS ==========

Model Accuracy on Test Set:

MAE: 0.7456

R^2: 0.8412

========== STATS ON PREDICTED G4 ==========

Mean G4: 11.91

Min G4: 0.30

Max G4: 18.42

Std G4: 3.09

Saved predictions to: predicted_G4_results.csv

5.5 Summary

The Student Performance Prediction System has been successfully implemented as a modular and automated pipeline. By excluding the final grade (G3) during prediction, it realistically forecasts G4_Predicted for early identification of at-risk students.

6. TESTING AND DEBUGGING

6.1 Overview

Testing is done to ensure that the Student Performance Prediction System works correctly, is error-free and produces reliable results. Debugging involves identifying, analyzing and fixing the errors encountered during the development and execution of the system.

6.3 Test Result

Student_ID	school	sex	age	G1	G2	G3	G4_Predicted
1	0	M	18	0	11	11	10.66
2	0	M	17	9	11	11	11.01
3	0	M	15	12	13	12	12.92
4	0	M	15	14	14	14	14.51
5	0	M	16	11	13	13	13.03
6	0	F	16	12	12	13	12.86
7	0	F	16	13	12	13	12.58
8	0	M	17	10	13	13	12.91
9	0	F	15	15	16	17	16.72
10	0	F	15	12	12	13	12.86

6.3 Types of Testing

6.3.1 Unit Testing

Each module was tested individually to check its functionality.

- preprocessing.py – Verified removal of missing values, duplicate records and conversion of categorical data using LabelEncoder().
- eda.py – Checked the generation of correlation heatmaps, histograms, boxplots and pairplots.

- feature_selection.py – Tested correlation calculation and selection of features with |correlation| >= 0.1.

- model.py – Verified data splitting, model training, prediction and calculation of MAE, MSE, RMSE and R^2 Score.

- visualization.py – Tested model comparison plots, residual plots and feature importance graphs.

- prediction.py – Verified the creation of the G4_Predicted column and export of results to predicted_G4_results.csv.

- Student.py – Checked dataset download and initial data display.

6.3.2 Integration Testing

Integration testing was carried out by running all modules together through main.py. This ensured proper data flow between modules and that the output of one module correctly served as the input to the next.

6.3.3 System Testing

The entire pipeline was executed end-to-end using the Student Performance dataset. This test confirmed that the system runs sequentially from data loading to final prediction without interruption.

6.3.4 Validation Testing

The predicted G4_Predicted values were compared with prior academic grades (G1 and G2). Since G1 and G2 showed a strong correlation with final performance, the predicted values followed a logical and reasonable trend, validating the model's output.

6.4 Debugging

Several errors were encountered during implementation and successfully resolved:

- Dataset Not Found: The system failed to locate the CSV file initially. This was fixed by adding the find_dataset() function in main.py to automatically search for the dataset in the local directory or KaggleHub cache.
- Categorical Encoding Errors: Some object-type columns caused errors during model training. This was resolved by applying LabelEncoder() to all categorical columns in the preprocessing stage.

- Plot Blocking Execution: Matplotlib plot windows were pausing the pipeline. This was fixed by using plt.close('all') after each visualization step to allow smooth sequential execution.

- Feature Mismatch in Prediction: Errors occurred due to using encoded and inconsistent features. This was corrected by reloading and preprocessing the data separately for the prediction step and excluding G3 while training the Random Forest model.

- Incorrect Display of Categorical Values: Earlier outputs showed values like 'school' as 0. This occurred as the data was already label-encoded before display. The approach was corrected to ensure proper handling of original values for result representation.

6.5 Test Results

All modules were tested successfully and passed the required test cases. The complete pipeline executed without any runtime errors, displayed all evaluation metrics and visualizations correctly, and generated predicted_G4_results.csv as expected. The Random Forest Regressor showed the best overall performance among the models tested.

6.6 Summary

Testing and debugging played a vital role in making the Student Performance Prediction System stable and reliable. Unit, integration, system and validation tests confirmed the correctness of each module and the system as a whole. The errors encountered were effectively resolved, resulting in a fully functional, automated pipeline that can accurately predict G4_Predicted for early identification of at-risk students.

7. CONCLUSION

The main objective of this project was to develop a Student Performance Prediction System using machine learning techniques to analyze and predict student academic performance. The system was successfully implemented using Python and open-source libraries such as Pandas, NumPy, Matplotlib, Seaborn and Scikit-learn.

The project followed a systematic approach, starting from data collection and preprocessing to exploratory data analysis, feature selection, model building, evaluation, prediction and visualization. The Student Performance dataset from the UCI Machine Learning Repository was used for this purpose. After cleaning, encoding and selecting relevant features, four regression models namely Linear Regression, Logistic Regression, Decision Tree Regressor and Random Forest Regressor were implemented and compared using MAE, MSE, RMSE and R^2 Score.

Among all the models tested, the Random Forest Regressor performed the best in terms of accuracy and reliability. Instead of predicting the final grade (G3) directly, the system excluded it during the prediction phase and generated a projected grade, referred to as G4_Predicted, using prior academic records and other attributes. This makes the system more practical for real-world use, as it enables the early identification of at-risk students and allows for timely academic intervention.

The entire pipeline was automated through main.py, making it modular, efficient and user-friendly. The final results were exported to predicted_G4_results.csv along with various visualizations, providing clear and meaningful insights for educators and institutions.

8. FUTURE ENHANCEMENT

8.1 Overview

Although the Student Performance Prediction System has successfully achieved its objectives, there is still room for improvement. With further development, the system can be made more accurate, scalable, user-friendly and better suited for real-world use in educational institutions.

8.2 Proposed Enhancements

- Use of Larger and More Diverse Datasets: The current dataset is limited to only two Portuguese schools. Using data from multiple schools, regions or academic years would improve the model's generalizability and make it more reliable across different educational setups.

- Implementation of Advanced Algorithms: More advanced techniques such as XGBoost, LightGBM, CatBoost, Support Vector Regression (SVR) or Artificial Neural Networks (ANN) can be explored to achieve higher prediction accuracy.

- Inclusion of Additional Attributes: Factors like student attendance, class participation, mental well-being, internet access, travel time and extracurricular activities can be added to provide a more holistic view of student performance.

- Feature Engineering: New derived features, such as the average of G1 and G2 or a combined study-effort score, can be created to help the model capture stronger patterns and improve its predictions.

- Development of a Web-Based Application: The system can be converted into an interactive web application using frameworks like Flask, Django or Streamlit. This would allow teachers and administrators to input student details and get instant predictions without running any code.

- Cloud Deployment: Deploying the application on cloud platforms such as Streamlit Cloud, Render or AWS would make it easily accessible from anywhere and support multiple users simultaneously.

- Integration with Student Management Systems: Linking the system with existing Student Management Systems would automate data collection, reduce manual work and allow for continuous performance monitoring.

- Mobile Application Development: A dedicated mobile application can be developed for teachers and students to easily track predicted performance on the go.

8.3 Summary

By implementing the above enhancements, the Student Performance Prediction System can be transformed from a standalone academic project into a complete, intelligent decision-support tool. With better accuracy, accessibility and interpretability, it would play a vital role in early identification of at-risk students, effective academic planning and ultimately, improved learning outcomes.

9. REFERENCES

Al-Barrak, M. A., & Al-Razgan, M. (2016). Predicting students final GPA using decision trees: A case study. *International Journal of Information and Education Technology*, *6*(7), 528–533. https://doi.org/10.7763/IJIET.2016.V6.745

Cortez, P., & Silva, A. M. G. (2008). Using data mining to predict secondary school student performance. In A. Brito & J. Teixeira (Eds.), *Proceedings of the 5th Future Business Technology Conference* (pp. 5–12). https://repositorium.sdum.uminho.pt/handle/1822/8024

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Hussain, S., Dahan, N. A., Ba-Alwib, F. M., & Ribata, N. (2018). Educational data mining and analysis of students' academic performance using WEKA. *Indonesian Journal of Electrical Engineering and Computer Science*, *9*(2), 447–459. https://doi.org/10.11591/ijeecs.v9.i2.pp447-459

Kotsiantis, S. B., Pierrakeas, C. J., & Pintelas, P. E. (2004). Predicting students' dropout-prone using machine learning techniques. *Journal of Computer Assisted Learning*, *20*(5), 374–383. https://doi.org/10.1111/j.1365-2729.2004.00098.x

McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Kaggle. (n.d.). *kagglehub* [Computer software]. GitHub. https://github.com/Kaggle/kagglehub

Montgomery, D. C., Peck, E. A., & Vining, G. G. (2012). *Introduction to linear regression analysis* (5th ed.). John Wiley & Sons.

Romero, C., & Ventura, S. (2010). Educational data mining: A review of the state of the art. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, *40*(6), 601–618. https://doi.org/10.1109/TSMCC.2010.2053532